Instituto Tecnológico de Costa Rica

Escuela de Computación

Lenguajes de programación

Proyecto #2 - Sistema de Eventos Comerciales

Código de curso: IC4700

Estudiante: Luis Trejos Rivera | 2022437816

Profesor: Allan Rodríguez Davila

Grupo: 60

I Semestre 2026

Contenido

Manual de usuario 4
Requisitos previos 4
Compilación 4
Ejecución 4
Uso del sistema 4
Pruebas de funcionalidad 6
Inicio del sistema 6
Opción 1a: Aplicar impuesto a compras 6
Opción 1b: Etiquetar eventos de alto valor 7
Opción 2a: Monto total 8
Opción 2b: Promedio por categoría y año 8
Opción 3a: Mes con mayor monto y día más activo 9
Opción 3b: Evento más antiguo y más reciente 9
Opción 3c: Resumen por intervalo 10
Opción 4: Búsqueda por rango de fechas 10
Validación de fecha inválida 11
Opción 5: Estadísticas y exportación 12
Descripción del problema 13
Contexto 13
Problema a resolver 13
Diseño del programa 13
Decisiones de diseño: 13
Algoritmos más importantes 14
Librerías usadas 15
System.IO: 15
System.Random: 15

Data.Time: 15
Data.List: 16
Data.Ord: 16
System.Directory: 16
Análisis de resultados 16
Objetivos alcanzados: 16
Objetivos no alcanzados: 17
Justificación de toma de decisiones 17
a. Funciones de orden superior utilizadas 17
b. Cómo se encadenan las funciones 20
Bitácora: 23

Manual de usuario

Requisitos previos

Para compilar y ejecutar el sistema se requiere tener instalado:

- **GHC** (Glasgow Haskell Compiler) versión 9.10.3 o superior
- **Stack** versión 3.7.1 o superior

Compilación

Desde la terminal, ubicarse en la carpeta raíz del proyecto y ejecutar:

“stack build”

La primera vez tardará varios minutos porque descarga las dependencias random, time y directory. Las compilaciones siguientes son más rápidas. También se puede hacer el stack run directamente ya que ese hace el mismo proceso junto a la ejecución.

Ejecución

“stack run”

El sistema arranca automáticamente, genera los primeros eventos y muestra el menú principal.

Uso del sistema

Menú principal ingresar el número de la opción deseada y presionar Enter:

```
==============================
  Sistema de Eventos PP2
==============================
1. Transformacion de eventos
2. Analisis de datos
3. Analisis temporal
4. Busqueda
5. Estadisticas
0. Salir
==============================

Seleccione una opcion:
```

1) Transformación de eventos:

a. Muestra las compras antes y después de aplicar el impuesto del 13%
b. Lista todos los eventos indicando cuáles superan el promedio de su categoría

2) Análisis de datos:

a. Muestra la suma total de todos los montos del sistema
b. Muestra el promedio de monto por categoría separado por año

3) Análisis temporal:

a. Muestra el mes/año con mayor monto acumulado y el día de la semana más activo
b. Muestra el evento más antiguo y el más reciente según su timestamp
c. Muestra cantidad y monto total agrupados por intervalos de valor

4) Búsqueda:

- Solicita dos fechas en formato YYYYMMDD y muestra los eventos en ese rango
- Ejemplo: inicio 20260101, fin 20271231

5) Estadísticas:

- Muestra el resumen general y ofrece exportar el reporte:
    a. Exporta a CSV,
    b. Exporta a JSON
    c. Exporta ambos
    d. No exporta nada

- Si decide exportar los archivos se guardan en la carpeta exports/ con nombre estadisticas_YYYY-MM-DD

0) Salir:

- Termina el programa. Los datos no son persistentes y se liberan de memoria.

Nota importante: al finalizar cada opción el sistema genera automáticamente entre 10 y 25 eventos nuevos que se agregan al sistema.

Pruebas de funcionalidad

Inicio del sistema

Al ejecutar stack run en la terminal, el sistema inicializa automáticamente y genera los primeros eventos aleatorios. Como se muestra en la Figura 1, el programa despliega el mensaje de inicio, indica la cantidad de eventos generados y presenta el menú principal con las cinco opciones disponibles más la opción de salir. El sistema queda listo para recibir la selección del usuario.

[Figura 1]

```
PS C:\Users\luist\OneDrive\Escritorio\LENG-P02\PP2> stack run
Iniciando sistema...
  [Se generaron 16 nuevos eventos]

==============================
  Sistema de Eventos PP2
==============================
1. Transformacion de eventos
2. Analisis de datos
3. Analisis temporal
4. Busqueda
5. Estadisticas
0. Salir
==============================

Seleccione una opcion:
```

Opción 1a: Aplicar impuesto a compras

Al seleccionar la opción 1 y luego la sub-opción a, el sistema identifica todos los eventos de categoría Compra existentes en memoria. Como se observa en la Figura 2, primero se despliegan los montos originales de cada compra y seguidamente se muestran los valores resultantes después de aplicar el impuesto de ventas del 13%, permitiendo verificar de forma clara la transformación realizada sobre cada evento.

[Figura 2]

```
Seleccione una opcion: 1

--- Transformacion de eventos ---
a. Aplicar impuesto a compras
b. Etiquetar eventos de alto valor
Seleccione: a

Compras encontradas: 2

  ANTES del impuesto:
  ID: 2936950 | Cat: Compra | Valor: 50686.25448082806 | TS: 20270418
  ID: 8771235 | Cat: Compra | Valor: 45839.147182064255 | TS: 20271206

  DESPUES del impuesto (13%):
  ID: 2936950 | Cat: Compra | Valor: 57275.467563335704 | TS: 20270418
  ID: 8771235 | Cat: Compra | Valor: 51798.2363157326 | TS: 20271206
  [Se generaron 21 nuevos eventos]
```

Opción 1b: Etiquetar eventos de alto valor

Al seleccionar la opción 1 y luego la sub-opción b, el sistema calcula dinámicamente el promedio de valor para cada categoría y compara cada evento contra el promedio de su propia categoría. Como se aprecia en la Figura 3, los eventos cuyo monto supera ese promedio reciben la etiqueta de alto valor, mientras que los que no lo superan se muestran sin etiqueta, permitiendo identificar fácilmente cuáles son los eventos más relevantes dentro de cada categoría.

[Figura 3]

```
Seleccione una opcion: 1

--- Transformacion de eventos ---
a. Aplicar impuesto a compras
b. Etiquetar eventos de alto valor
Seleccione: b

Eventos etiquetados por alto valor:
  ID: 3352374 | Cat: Seguimiento | Valor: 29769.43579316986
  ID: 5138053 | Cat: Seguimiento | Valor: 2505.987148862968
  ID: 1339138 | Cat: Visualizacion | Valor: 22181.041553425373
  ID: 2280051 | Cat: Devolucion | Valor: 28911.052322573447
  ID: 2443637 | Cat: Apartado | Valor: 27448.147107171586
  ID: 2936950 | Cat: Compra | Valor: 57275.467563335704 [*** ALTO VALOR ***]
  ID: 8276534 | Cat: Visualizacion | Valor: 50093.11898621274 [*** ALTO VALOR ***]
  ID: 4446197 | Cat: Apartado | Valor: 62789.9654375328 [*** ALTO VALOR ***]
  ID: 5223052 | Cat: Apartado | Valor: 39813.17289138974
  ID: 8484538 | Cat: Devolucion | Valor: 3652.089214677916
  ID: 8082942 | Cat: Visualizacion | Valor: 39848.10652479881 [*** ALTO VALOR ***]
  ID: 1141668 | Cat: Devolucion | Valor: 41683.04264310962 [*** ALTO VALOR ***]
  ID: 2803816 | Cat: Seguimiento | Valor: 23511.198216849458
  ID: 8047704 | Cat: Visualizacion | Valor: 71231.98628008012 [*** ALTO VALOR ***]
  ID: 8921930 | Cat: Apartado | Valor: 68462.50417254487 [*** ALTO VALOR ***]
  ID: 8771235 | Cat: Compra | Valor: 51798.2363157326 [*** ALTO VALOR ***]
  ID: 7739242 | Cat: Apartado | Valor: 19144.20133004517
  ID: 1137204 | Cat: Seguimiento | Valor: 44286.74982256207 [*** ALTO VALOR ***]
  ID: 4965806 | Cat: Devolucion | Valor: 40367.4590180865 [*** ALTO VALOR ***]
  ID: 1214665 | Cat: Seguimiento | Valor: 52072.868416124234 [*** ALTO VALOR ***]
  ID: 7354425 | Cat: Devolucion | Valor: 71233.29888710225 [*** ALTO VALOR ***]
  ID: 2858091 | Cat: Compra | Valor: 15236.726480505678
```

Opción 2a: Monto total

Al seleccionar la opción 2 y luego la sub-opción a, el sistema suma los montos de todos los eventos almacenados en memoria utilizando una operación de agregación funcional. Como se muestra en la Figura 4, el resultado se despliega en pantalla como un único valor que representa el total acumulado de todos los eventos del sistema en ese momento.

[Figura 4]

```
Seleccione una opcion: 2

--- Analisis de datos ---
a. Monto total
b. Promedio por categoria y anio
Seleccione: a

  Monto total: 2129894.214367583
  [Se generaron 14 nuevos eventos]
```

Opción 2b: Promedio por categoría y año

Al seleccionar la opción 2 y luego la sub-opción b, el sistema agrupa los eventos por categoría y año, calcula el promedio de valor para cada combinación y muestra únicamente los grupos que tienen al menos un evento. Como se observa en la Figura 5, los resultados se presentan indicando la categoría, el año y el promedio correspondiente, separando correctamente períodos distintos para evitar mezclar datos de años diferentes.

[Figura 5]

```
Seleccione una opcion: 2

--- Analisis de datos ---
a. Monto total
b. Promedio por categoria y anio
Seleccione: b

  Promedio por categoria y anio:
    Visualizacion | 2026 | Promedio: 4460.882167517299
    Visualizacion | 2027 | Promedio: 35179.64326492949
    Apartado | 2026 | Promedio: 33994.28482854643
    Apartado | 2027 | Promedio: 37105.82174016087
    Compra | 2026 | Promedio: 44214.39441428123
    Compra | 2027 | Promedio: 32343.45808134293
    Devolucion | 2026 | Promedio: 54233.892707844905
    Devolucion | 2027 | Promedio: 45952.961214377385
    Seguimiento | 2026 | Promedio: 29819.690816517006
    Seguimiento | 2027 | Promedio: 34057.32297809349
  [Se generaron 14 nuevos eventos]
```

Opción 3a: Mes con mayor monto y día más activo

Al seleccionar la opción 3 y luego la sub-opción a, el sistema analiza todos los eventos y determina el mes y año con mayor monto acumulado, así como el día de la semana con mayor cantidad de eventos registrados. Como se aprecia en la Figura 6, ambos resultados se presentan en pantalla indicando el período ganador con su monto total y el día de la semana con el conteo de eventos correspondiente.

[Figura 6]

```
Seleccione una opcion: 3

--- Analisis Temporal ---
a. Mes con mayor monto y dia mas activo
b. Evento mas antiguo y reciente
c. Resumen por intervalo
Seleccione: a

  Mes con mayor monto: 12/2027 | Total: 366182.02102067583
  Dia de semana mas activo: Tuesday (23 eventos)
  [Se generaron 23 nuevos eventos]
```

Opción 3b: Evento más antiguo y más reciente

Al seleccionar la opción 3 y luego la sub-opción b, el sistema recorre todos los timestamps almacenados y determina cuál evento tiene la fecha más antigua y cuál tiene la fecha más reciente. Como se muestra en la Figura 7, ambos eventos se presentan con su información completa, permitiendo conocer el rango temporal que abarca el conjunto de datos actual del sistema.

[Figura 7]

```
Seleccione una opcion: 3

--- Analisis Temporal ---
a. Mes con mayor monto y dia mas activo
b. Evento mas antiguo y reciente
c. Resumen por intervalo
Seleccione: b

  Evento mas antiguo:
  ID: 4299451 | Cat: Apartado | Valor: 9051.282540834593 | TS: 20260508
  Evento mas reciente:
  ID: 8484538 | Cat: Devolucion | Valor: 3652.089214677916 | TS: 20280506
  [Se generaron 14 nuevos eventos]
```

Opción 3c: Resumen por intervalo

Al seleccionar la opción 3 y luego la sub-opción c, el sistema clasifica todos los eventos en tres rangos de valor predefinidos y calcula para cada rango la cantidad de eventos y el monto total acumulado. Como se observa en la Figura 8, los intervalos cubren los rangos de 500 a 10.000, de 10.001 a 30.000 y de 30.001 a 75.000, ofreciendo una visión general de la distribución de montos en el sistema.

[Figura 8]

```
Seleccione una opcion: 3

--- Analisis Temporal ---
a. Mes con mayor monto y dia mas activo
b. Evento mas antiguo y reciente
c. Resumen por intervalo
Seleccione: c

  Resumen por intervalo de monto:
    [  500  - 10000 ]  Cantidad: 14  |  Monto total: 71803.62472223409
    [10001  - 30000 ]  Cantidad: 44  |  Monto total: 878193.9816567932
    [30001  - 75000 ]  Cantidad: 92  |  Monto total: 4620484.485766883
  [Se generaron 18 nuevos eventos]
```

Opción 4: Búsqueda por rango de fechas

Al seleccionar la opción 4, el sistema solicita una fecha de inicio y una fecha de fin en formato YYYYMMDD. Como se muestra en la Figura 9, una vez ingresadas las fechas el sistema valida que el formato sea correcto y que la fecha de inicio no sea mayor que la fecha de fin, retornando todos los eventos cuyo timestamp se encuentre dentro del rango especificado. En caso de que no existan eventos en ese período, el sistema lo indica con un mensaje apropiado.

[Figura 9]

```
Seleccione una opcion: 4

--- Busqueda por rango de fechas ---

  Ingrese fecha inicio (YYYYMMDD, ej: 20260101):
  > 20260101
  Ingrese fecha fin (YYYYMMDD, ej: 20271231):
  > 20261201

  Eventos encontrados: 49
  ID: 5138053 | Cat: Seguimiento | Valor: 2505.987148862968 | TS: 20260831
  ID: 4446197 | Cat: Apartado | Valor: 62789.9654375328 | TS: 20261112
  ID: 1141668 | Cat: Devolucion | Valor: 41683.04264310962 | TS: 20261121
  ID: 2803816 | Cat: Seguimiento | Valor: 23511.198216849458 | TS: 20261201
  ID: 2858091 | Cat: Compra | Valor: 15236.726480505678 | TS: 20260822
  ID: 4367751 | Cat: Visualizacion | Valor: 4460.882167517299 | TS: 20261108
  ID: 4891894 | Cat: Seguimiento | Valor: 43779.37861315278 | TS: 20260714
  ID: 3286067 | Cat: Devolucion | Valor: 66784.7427725802 | TS: 20261015
  ID: 7952999 | Cat: Compra | Valor: 50836.57813637913 | TS: 20260925
  ID: 2923871 | Cat: Apartado | Valor: 55300.05035049912 | TS: 20260830
  ID: 6197946 | Cat: Compra | Valor: 42334.412433049874 | TS: 20260619
  ID: 5826950 | Cat: Seguimiento | Valor: 4940.960289689163 | TS: 20261123
  ID: 2514829 | Cat: Compra | Valor: 56126.78313419057 | TS: 20260609
  ID: 4299451 | Cat: Apartado | Valor: 9051.282540834593 | TS: 20260508
  ID: 8072638 | Cat: Compra | Valor: 39487.81342688323 | TS: 20260926
  ID: 5241973 | Cat: Visualizacion | Valor: 51526.75819717217 | TS: 20260627
  ID: 500574 | Cat: Apartado | Valor: 38380.65393504048 | TS: 20260719
```

Validación de fecha inválida

Como parte del manejo de errores de la búsqueda, si el usuario ingresa una fecha con formato incorrecto o caracteres no numéricos, el sistema detecta la inconsistencia y despliega un mensaje de error sin interrumpir la ejecución. Como se aprecia en la Figura 10, el programa continúa funcionando normalmente después del error, volviendo al menú principal para que el usuario pueda intentarlo nuevamente.

[Figura 10]

```
Seleccione una opcion: 4

--- Busqueda por rango de fechas ---

  Ingrese fecha inicio (YYYYMMDD, ej: 20260101):
  > 2025-
  Ingrese fecha fin (YYYYMMDD, ej: 20271231):
  > 600000

  ERROR: Formato de fecha invalido. Use YYYYMMDD.
  [Se generaron 18 nuevos eventos]
```

Opción 5: Estadísticas y exportación

Al seleccionar la opción 5, el sistema despliega el resumen general de estadísticas que incluye la cantidad de eventos por categoría, el evento con el monto más alto, el evento con el monto más bajo y la fecha con mayor cantidad de eventos registrados. Seguidamente ofrece la posibilidad de exportar el reporte. Como se muestra en la Figura 11, al seleccionar la opción de exportar ambos formatos, el sistema genera los archivos correspondientes en la carpeta exports con un nombre único que incluye la fecha actual, confirmando la operación en pantalla.

[Figura 11]

```
Seleccione una opcion: 5

--- Estadisticas ---

  --- Cantidad por categoria ---
    Visualizacion: 9
    Apartado: 2
    Compra: 6
    Devolucion: 4
    Seguimiento: 1

  --- Evento con monto mas alto ---
  ID: 1918990 | Cat: Apartado | Valor: 72193.45227162058 | TS: 20270905

  --- Evento con monto mas bajo ---
  ID: 4664229 | Cat: Visualizacion | Valor: 3659.951714388073 | TS: 20270915

  --- Dia con mayor cantidad de eventos ---
    Fecha: 20270213 | Eventos: 1

  Exportar reporte:
  a. CSV
  b. JSON
  c. Ambos
  d. No exportar
  Seleccione: c
  Exitoso: Estadisticas exportadas a exports/estadisticas_2026-05-07_03-57-06.csv
  Exitoso: Estadisticas exportadas a exports/estadisticas_2026-05-07_03-57-06.json
  [Se generaron 15 nuevos eventos]
```

Descripción del problema

Contexto

En la actualidad, las plataformas de comercio electrónico generan grandes volúmenes de datos en tiempo real producto de las interacciones de sus usuarios. Cada acción relevante dentro de la plataforma como una compra, una visualización de producto, un apartado, una devolución o un seguimiento, constituye un evento que contiene información valiosa para la toma de decisiones comerciales.

El procesamiento eficiente de estos eventos requiere sistemas capaces de filtrar, transformar y analizar datos de forma estructurada, preferiblemente bajo paradigmas de programación que garanticen la consistencia y la ausencia de efectos secundarios no deseados.

Problema a resolver

Se requiere desarrollar un sistema en consola que simule el monitoreo de actividad de usuarios dentro de una plataforma de comercio electrónico. El sistema debe ser capaz de:

- **Generar eventos dinámicamente** de forma automática, simulando el flujo continuo de actividad de usuarios en tiempo real.
- **Almacenar los eventos en memoria** durante la sesión, tratándolos como una estructura de datos funcional inmutable.
- **Transformar los eventos** aplicando reglas de negocio como el impuesto de ventas y la identificación de eventos de alto valor.
- **Analizar los datos** mediante operaciones de agregación, filtrado y comparación temporal.
- **Exportar reportes** en formatos estándar como CSV y JSON para su uso externo.
- **Validar la integridad** de los datos reportando inconsistencias como montos inválidos.

Diseño del programa

Decisiones de diseño:

Tipos algebraicos como estructura central

El sistema modela los datos usando dos tipos algebraicos. Categoría es un tipo suma un valor solo puede ser una de las cinco opciones definidas, lo que elimina la posibilidad de categorías inválidas en tiempo de compilación. Evento es un tipo producto, agrupa los cuatro campos obligatorios en una sola estructura. Esta decisión

garantiza que cualquier evento en el sistema siempre tiene todos sus campos, nunca parciales ni nulos.

Estado del sistema como parámetro

En lugar de usar variables globales o estado mutable, la lista de eventos se pasa como parámetro en cada llamada recursiva de menuLoop. Cada vez que se generan eventos nuevos, se crea una lista nueva que combina los eventos anteriores con los nuevos y se pasa a la siguiente iteración. Esto respeta la inmutabilidad de Haskell y hace que el flujo de datos sea explícito y predecible.

Timestamp como entero YYYYMMDD

Se decidió representar las fechas como enteros en formato YYYYMMDD en lugar de usar el tipo Day directamente. Esto simplifica el almacenamiento, la comparación y la búsqueda por rango. Comparar dos enteros es directo sin necesidad de conversiones. La conversión a Day solo se realiza cuando se necesita calcular el día de la semana, usando timestampADay.

Generación dinámica al finalizar cada opción

El enunciado requiere que el sistema se alimente de nuevos eventos después de cada operación. Esto se implementó llamando a generarEventos al final de cada rama del menú antes de recursar en menuLoop. De esta forma el sistema simula un flujo continuo de datos sin interrumpir la experiencia del usuario.

Exportación con nombre único por fecha

Para evitar que cada exportación sobreescriba la anterior, el nombre del archivo incluye la fecha actual en formato YYYY-MM-DD_hh_mm_ss. Esto permite generar múltiples reportes en la misma sesión sin perder los anteriores.

Algoritmos más importantes

Etiquetado de alto valor

La función etiquetarAltoValor opera en dos pasos. Primero calcula el promedio de valor para cada una de las cinco categorías usando map sobre la lista de categorías, filtrando los eventos de cada una con filter y calculando el promedio con foldl. Luego aplica map sobre todos los eventos, buscando el promedio correspondiente a su categoría y comparando su valor contra ese promedio. El resultado es una lista de tuplas (Evento, Bool) donde el booleano indica si el evento supera el promedio de su categoría.

Mes con mayor monto separado por año

La función mesMayorMonto usa una list comprehension que genera todas las combinaciones posibles de año y mes dentro del rango del proyecto. Para cada

combinación filtra los eventos que corresponden a ese período, suma sus montos con foldl y descarta las combinaciones vacías. Finalmente aplica foldl1 para encontrar la combinación con el mayor acumulado.

Búsqueda por rango de fechas

La función buscarPorRango aplica filter sobre la lista de eventos comparando el timestamp de cada evento contra los límites ingresados. La validación del formato se realiza con reads, que intenta parsear el String a entero de forma segura; si falla retorna una lista vacía que se detecta con pattern matching, evitando excepciones en tiempo de ejecución.

Resumen por intervalo

La función resumenPorIntervalo define los tres intervalos como una lista de tuplas donde cada tupla contiene el nombre del intervalo y una función lambda que actúa como predicado. Luego aplica mapM_ sobre esa lista, filtrando los eventos con filter (cond . valor), donde la composición de funciones. primero extrae el valor del evento y luego le aplica la condición del intervalo.

Librerías usadas

System.IO:

Funciones usadas: hSetBuffering, stdout, BufferMode(..)

Se usa para configurar el modo de buffer de la salida estándar. Al establecer NoBuffering con hSetBuffering stdout NoBuffering al inicio del programa, se garantiza que el texto aparezca en consola inmediatamente sin esperar a que el buffer se llene. Sin esto, el menú y los mensajes podrían no mostrarse en el momento correcto, especialmente el prompt Seleccione una opcion: que aparece sin salto de línea al final.

System.Random:

Funciones usadas: randomRIO

Permite generar números aleatorios dentro de un rango dado. Se usa en generarEvento para producir valores aleatorios para el ID, la categoría, el monto y los días extra del timestamp. randomRIO (min, max) vive en IO porque cada llamada produce un resultado diferente, lo que rompe la pureza funcional, por eso se usa con <- dentro de bloques do.

Data.Time:

Funciones usadas: getCurrentTime, utctDay, toGregorian, fromGregorian, Day, dayOfWeek

Maneja todo lo relacionado con fechas. getCurrentTime obtiene la fecha y hora actual del sistema. utctDay extrae solo la parte de fecha.uctDayTime para poder obtener los minutos y segundos. toGregorian convierte un Day a una tupla (año, mes, día) que se usa para construir el timestamp YYYYMMDD. fromGregorian hace la conversión inversa, construyendo un Day desde año, mes y día, necesario para calcular fechas futuras. dayOfWeek retorna el día de la semana de un Day, usado en diaSemanaActivo.

Data.List:

Funciones usadas: intercalate, maximumBy

intercalate une una lista de Strings con un separador entre cada elemento, se usa para construir el contenido de los archivos CSV y JSON, por ejemplo, intercalate "," ["a","b","c"] produce "a,b,c". maximumBy encuentra el elemento máximo de una lista usando una función de comparación personalizada, se usa en eventoMayor y eventoMenor para encontrar los eventos con mayor y menor valor sin escribir un foldl1 manual.

Data.Ord:

Funciones usadas: comparing, Down(..)

comparing construye una función de comparación a partir de un selector de campo; comparing valor le dice a maximumBy que compare los eventos por su campo valor. Down es un wrapper que invierte el orden natural de comparación; comparing (Down. valor) permite encontrar el mínimo usando maximumBy sin necesidad de una función separada para mínimos.

System.Directory:

Funciones usadas: createDirectoryIfMissing

Crea directorios en el sistema de archivos. Se usa antes de escribir los archivos de exportación para garantizar que la carpeta exports/ exista. El primer parámetro True indica que debe crear también las carpetas intermedias si no existen, evitando errores cuando la carpeta no ha sido creada previamente.

Análisis de resultados

Objetivos alcanzados:

Estructura de datos funcional: Se implementó exitosamente el tipo algebraico Evento con sus cuatro campos obligatorios y el tipo suma Categoria con las cinco categorías requeridas. El manejo del estado mediante listas inmutables pasadas como parámetro cumple con el paradigma funcional solicitado.

Generación dinámica de eventos: El sistema genera entre 10 y 25 eventos aleatorios al finalizar cada opción del menú, con valores dentro de los rangos especificados en las instrucciones. La validación de montos está implementada y reporta inconsistencias en consola cuando se detectan.

Transformación de eventos: Se implementaron ambas transformaciones requeridas. El impuesto del 13% se aplica correctamente solo a eventos de categoría Compra, mostrando el estado antes y después de la transformación. El etiquetado de alto valor calcula dinámicamente el promedio por categoría y marca los eventos que lo superan.

Análisis de datos: El monto total y el promedio por categoría y año funcionan correctamente. El promedio separa los datos por año para que períodos distintos no se mezclen en el cálculo.

Análisis temporal: Se implementaron las tres sub-opciones: mes con mayor monto separado por año, evento más antiguo y reciente, y resumen por intervalos de valor con cantidad y monto total.

Búsqueda por rango de fechas: La búsqueda valida el formato de entrada, verifica que la fecha inicio no sea mayor que la fecha fin, y retorna los eventos dentro del rango especificado.

Estadísticas y exportación: El resumen general muestra cantidad por categoría, evento de mayor y menor valor, y el día con más eventos. La exportación genera archivos CSV y JSON con nombre único por fecha en la carpeta exports/.

Documentación: Se documentaron todas las funciones con objetivo, entradas, salidas y restricciones. El repositorio en GitHub refleja un desarrollo incremental con commits por funcionalidad.

Objetivos no alcanzados:

Persistencia de datos El enunciado ofrece 7.5 puntos adicionales por implementar persistencia mediante archivo o base de datos, de forma que los datos se mantengan entre sesiones. Este objetivo no se implementó debido a las limitaciones de tiempo del desarrollo. Los datos del sistema son volátiles y se liberan al cerrar el programa.

Justificación de toma de decisiones

a. Funciones de orden superior utilizadas

Las funciones de orden superior son funciones que reciben otras funciones como parámetro o las retornan como resultado. En este proyecto se usaron extensamente como mecanismo principal de procesamiento de datos.

Map:

Aplica una función a cada elemento de una lista y retorna una lista nueva del mismo tamaño. Se usa en múltiples partes del sistema:

En transformarImpuestos para aplicar el impuesto a todos los eventos:

[Figura 12]

```
transformarImpuestos :: [Evento] -> [Evento]
transformarImpuestos = map aplicarImpuesto
```

En etiquetarAltoValor para calcular el promedio de cada categoría:

[Figura 13]

```
etiquetarAltoValor :: [Evento] -> [(Evento, Bool)]
etiquetarAltoValor eventos =
  let categorias = [Visualizacion, Apartado, Compra, Devolucion, Seguimiento]
      -- Para cada categoría calcula su promedio y lo asocia en una tupla
      promedios  = map (\cat ->
                     let evsCat = eventosPorCategoria cat eventos
                     in (cat, promedioValor evsCat)
                   ) categorias
```

La ventaja de map sobre un loop tradicional es que no muta la lista original, siempre produce una lista nueva, garantizando inmutabilidad.

filter:

Retorna una sublista con solo los elementos que cumplen un predicado dado. Se usa en búsqueda, análisis y transformaciones:

En buscarPorRango para encontrar eventos dentro de un rango de fechas:

[Figura 14]

```
buscarPorRango :: Int -> Int -> [Evento] -> [Evento]
buscarPorRango inicio fin = filter (\e -> timestamp e >= inicio && timestamp e <= fin)
```

En eventosPorCategoria para aislar eventos de una categoría específica:

[Figura 15]

```
eventosPorCategoria :: Categoria -> [Evento] -> [Evento]
eventosPorCategoria cat = filter (\e -> categoria e == cat)
```

foldl:

Reduce una lista a un único valor acumulando un resultado de izquierda a derecha. Se usa para todas las operaciones de suma y búsqueda de máximos:

En montoTotal para sumar todos los montos:

[Figura 16]

```
montoTotal :: [Evento] -> Double
montoTotal = foldl (\acc e -> acc + valor e) 0
```

El primer argumento es una función lambda donde acc es el acumulador que empieza en 0 y en cada paso suma el valor del evento actual.

foldl1:

Igual que foldl pero usa el primer elemento como valor inicial, sin necesitar un valor de arranque externo. Se usa cuando se busca el máximo o mínimo dentro de una lista:

En mesMayorMonto para encontrar el período con mayor acumulado:

[Figura 17]

```
mesMayorMonto :: [Evento] -> (Int, Int, Double)
mesMayorMonto [] = (0, 0, 0)
mesMayorMonto eventos =
  let anios   = [2025, 2026, 2027, 2028]
      meses   = [1..12]
      -- List comprehension: genera tuplas (año, mes, total) para cada período con eventos
      totales = [ (anio, mes, total)
                | anio <- anios
                , mes  <- meses
                , let evs = filter (\e -> anioDeTimestamp (timestamp e) == anio
                                       && mesDeTimestamp  (timestamp e) == mes) eventos
                , not (null evs)
                , let total = foldl (\acc e -> acc + valor e) 0 evs
                ]
  in if null totales then (0, 0, 0)
     else foldl1 (\(ba, bm, bt) (a, m, t) ->
                  if t > bt then (a, m, t) else (ba, bm, bt)) totales
```

mapM y mapM_:

Son versiones de map para funciones que viven en IO. mapM recolecta los resultados en una lista, mapM_ descarta los resultados cuando solo importa el efecto.

En generarEventos para generar múltiples eventos en IO:

[Figura 18]

```
generarEventos :: [Evento] -> IO [Evento]
generarEventos existentes = do
  cantidad <- randomRIO (10, 25 :: Int)
  nuevos   <- mapM (const generarEvento) [1..cantidad]
  putStrLn $ "  [Se generaron " ++ show cantidad ++ " nuevos eventos]"
  return (existentes ++ nuevos)
```

En mostrarEtiquetados para imprimir cada evento:

[Figura 19]

```
mostrarEtiquetados :: [(Evento, Bool)] -> IO ()
mostrarEtiquetados = mapM_ mostrar
  where
    mostrar (e, alto) = putStrLn $
      "  ID: " ++ show (eventoId e) ++
      " | Cat: " ++ show (categoria e) ++
      " | Valor: " ++ show (valor e) ++
      if alto then " [*** ALTO VALOR ***]" else ""
```

maximumBy:

Encuentra el elemento máximo de una lista usando una función de comparación personalizada. Se usa junto con comparing para encontrar eventos extremos:

[Figura 20]

```
eventoMayor :: [Evento] -> Evento
eventoMayor = maximumBy (comparing valor)
```

Down invierte el orden para encontrar el mínimo usando la misma función.

b. Cómo se encadenan las funciones

El encadenamiento de funciones es uno de los pilares del paradigma funcional. En este proyecto se usaron tres mecanismos principales:

Composición con el operador .:

El operador . encadena dos funciones de forma que la salida de la segunda es la entrada de la primera. Se lee de derecha a izquierda:

[Figura 21]

```
  mapM_ (\(nombre, cond) -> do
    let evs   = filter (cond . valor) eventos
        total = foldl (\acc e -> acc + valor e) 0 evs
        cant  = length evs
```

Esto significa: primero aplicar valor al evento para extraer el monto, luego aplicar cond al monto resultante. Es equivalente a filter (\e -> cond (valor e)) eventos, pero más conciso.

[Figura 22]

```
eventoMayor :: [Evento] -> Evento
eventoMayor = maximumBy (comparing valor)
```

Aquí se encadenan tres funciones: primero valor extrae el monto, luego Down lo envuelve para invertir el orden, luego comparing construye la función de comparación.

Encadenamiento en bloques do:

Dentro de los bloques do, las acciones de IO se encadenan en secuencia. El resultado de una acción puede usarse en las siguientes mediante <-:

[Figura 23]

```
generarEvento :: IO Evento
generarEvento = do
  eid      <- randomRIO (0, 9000000 :: Int)
  catIdx   <- randomRIO (0, 4 :: Int)
  val      <- randomRIO (500.0, 75000.0 :: Double)
  hoy      <- getCurrentTime
  let diaHoy = utctDay hoy
  diasExtra <- randomRIO (0, 730 :: Int)
  let diaFuturo = toGregorian (addDaysSimple diasExtra diaHoy)
  let ts        = fromGregorian2Int diaFuturo
  -- Validación: monto debe ser mayor a cero; se reporta y corrige si no lo es
  valFinal <- if val <= 0
              then do
                putStrLn $ "  [INCONSISTENCIA] Evento " ++ show eid ++
                           " tenia monto invalido, corregido a 500.0"
                return 500.0
              else return val
  return Evento
    { eventoId  = eid
    , categoria = intACategoria catIdx
    , valor     = valFinal
    , timestamp = ts
    }
```

Cada línea con <- extrae el valor del contexto IO y lo hace disponible para las siguientes líneas. Las líneas con let calculan valores puros sin efectos.

Encadenamiento funcional con $:

El operador $ aplica una función al resultado de una expresión, evitando paréntesis anidados. Se lee de derecha a izquierda:

[Figura 24]

```
then do
  putStrLn $ "  [INCONSISTENCIA] Evento " ++ show eid ++
             " tenia monto invalido, corregido a 500.0"
  return 500.0
else return val
```

Es equivalente a putStrLn ("Total: " ++ show (montoTotal eventos)) pero más legible cuando las expresiones son largas.

Pipeline de transformaciones:

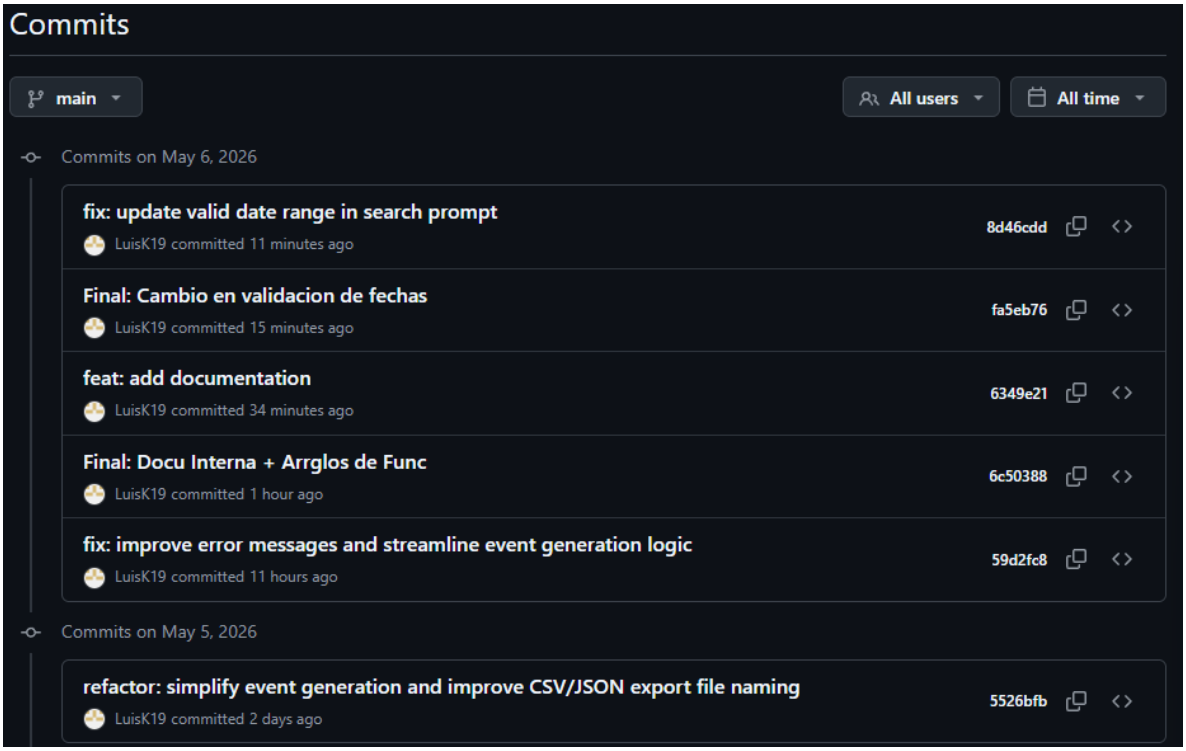
El flujo completo de procesamiento de datos en el sistema sigue un pipeline funcional. Por ejemplo, al calcular el promedio por categoría y año:

[Figura 25]

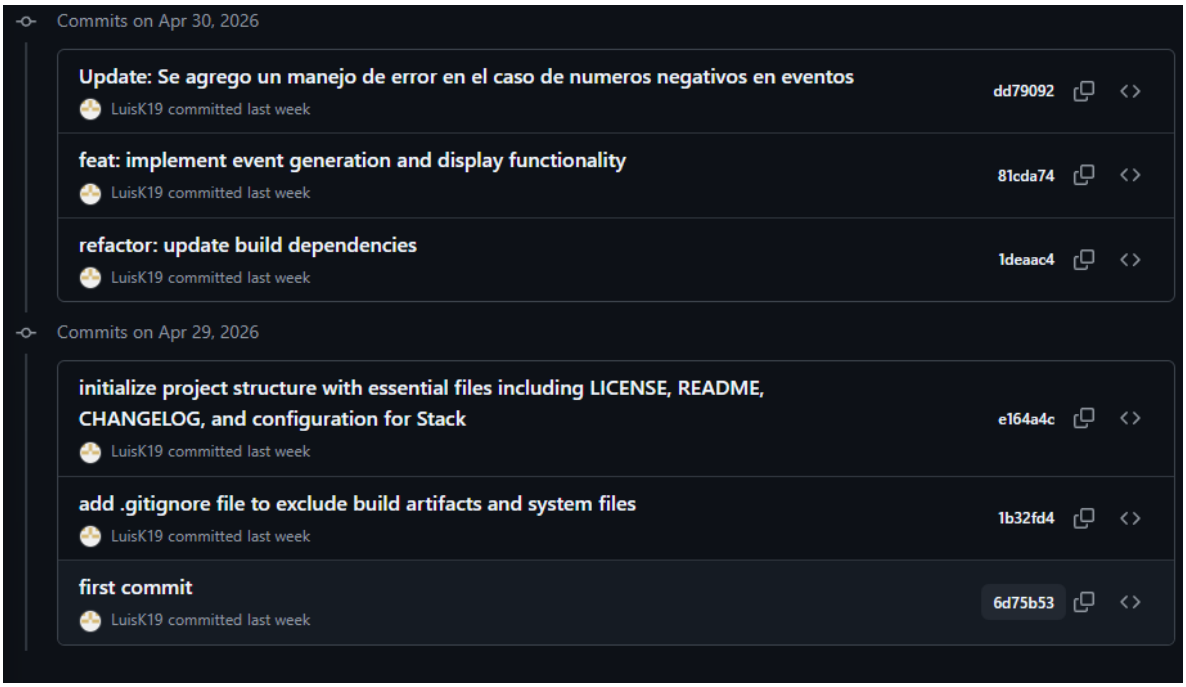
```
promedioPorCategoriaAnio eventos =
  let categorias = [Visualizacion, Apartado, Compra, Devolucion, Seguimiento]
      anios      = [2025, 2026, 2027]
      resultado  = [ (cat, anio, promedioValor evsFiltrados)
                   | cat  <- categorias
                   , anio <- anios
                   , let evsFiltrados = filter
                           (\e -> categoria e == cat &&
                                  anioDeTimestamp (timestamp e) == anio)
                           eventos
                   , not (null evsFiltrados)
                   ]
  in resultado
```

La list comprehension encadena iteración, filtrado y transformación en una sola expresión declarativa, sin variables mutables ni loops explícitos.

Bitácora:


Commits
main
All users
All time
Commits on May 6, 2026
fix: update valid date range in search prompt
LuisK19 committed 11 minutes ago
8d46cdd
Final: Cambio en validacion de fechas
LuisK19 committed 15 minutes ago
fa5eb76
feat: add documentation
LuisK19 committed 34 minutes ago
6349e21
Final: Docu Interna + Arrglos de Func
LuisK19 committed 1 hour ago
6c50388
fix: improve error messages and streamline event generation logic
LuisK19 committed 11 hours ago
59d2fc8
Commits on May 5, 2026
refactor: simplify event generation and improve CSV/JSON export file naming
LuisK19 committed 2 days ago
5526bfb

Commits on May 4, 2026
update: nuevos metodos CSV and JSON export
LuisK19 committed 2 days ago
39a9021
feat: add CSV and JSON export functionality for event statistics
LuisK19 committed 2 days ago
4a79357
feat: add temporal analysis functionality
LuisK19 committed 3 days ago
6d739cc
Commits on May 3, 2026
feat: add event transformation and analysis functionality
LuisK19 committed 3 days ago
f8a6173


Commits on Apr 30, 2026
Update: Se agrego un manejo de error en el caso de numeros negativos en eventos
LuisK19 committed last week
dd79092
feat: implement event generation and display functionality
LuisK19 committed last week
81cda74
refactor: update build dependencies
LuisK19 committed last week
1deaac4
Commits on Apr 29, 2026
initialize project structure with essential files including LICENSE, README, CHANGELOG, and configuration for Stack
LuisK19 committed last week
e164a4c
add .gitignore file to exclude build artifacts and system files
LuisK19 committed last week
1b32fd4
first commit
LuisK19 committed last week
6d75b53